1. Defining Problem Statement and Analysing basic metrics.

ANS:-

```
print("Total Entries:", simulated_df.shape[0])

print("\nContent Type Counts:\n", simulated_df['type'].value_counts())

print("\nRelease Year Range:", simulated_df['release_year'].min(), "-", simulated_df['release_year'].max())

print("\nTop 5 Countries:\n", simulated_df['country'].value_counts().head(5))

print("\nTop 5 Genres:\n", simulated_df['listed_in'].value_counts().head(5))
```

2. Observations on the shape of data, data types of all the attributes, conversion of categorical attributes to 'category' (If required), missing value detection, statistical summary.

   ANS:-

```
print("\nData Types:\n", simulated_df.dtypes)
print("\nMissing Values:\n", simulated_df.isnull().sum())

# Convert to categorical
cat_cols = ['type', 'rating', 'country', 'listed_in']
for col in cat_cols:
    simulated_df[col] = simulated_df[col].astype('category')

print("\nCategorical Columns Converted:", cat_cols)
print("\nStatistical Summary:\n", simulated_df.describe(include='all'))
```

3. Non-Graphical Analysis: Value counts and unique attributes .

   ANS:-

```
print("\nUnique Values per Column:\n", simulated_df.nunique())
print("\nRating Distribution:\n", simulated_df['rating'].value_counts())
print("\nTop 5 Directors:\n", simulated_df['director'].value_counts().head())
print("\nTop 5 Casts:\n", simulated_df['cast'].value_counts().head())
print("\nTop 5 Durations:\n", simulated_df['duration'].value_counts().head())
```

   4. Visual Analysis - Univariate, Bivariate after pre-processing of the data

   Note: Pre-processing involves unnesting of the data in columns like Actor, Director, Country

   4.1 For continuous variable(s): Distplot, countplot, histogram for univariate analysis.

   ANS:-

```
# Distribution of release years
plt.figure(figsize=(10,5))
sns.histplot(df['release_year'], bins=40, kde=True)
plt.title('Distribution of Release Years')
plt.show()

# Movie duration extraction
df['duration_int'] = df['duration'].str.extract('(\d+)').astype(float)
df['duration_type'] = df['duration'].str.extract('([a-zA-Z]+)')
```

```
# Histogram of movie durations
plt.figure(figsize=(10,5))
sns.histplot(df[df['type'] == 'Movie']['duration_int'], bins=30, kde=True)
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (minutes)')
plt.show()
```

4.2 For categorical variable(s): Boxplot.

ANS:-

```
plt.figure(figsize=(8,5))
sns.boxplot(x='type', y='duration_int', data=df)
plt.title('Boxplot of Duration by Type')
plt.show()
```

4.3 For correlation: Heatmaps, Pairplots.

ANS:-

```
# Only numeric columns
corr_data = df[['release_year', 'duration_int']].dropna()

# Correlation heatmap
plt.figure(figsize=(5,4))
sns.heatmap(corr_data.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

5. Missing Value & Outlier check (Treatment optional).

ANS:-

```
movie_durations = simulated_df[simulated_df['type'] == 'Movie']['duration_int']
Q1 = movie_durations.quantile(0.25)
Q3 = movie_durations.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = movie_durations[(movie_durations < lower_bound) | (movie_durations > upper_bound)]

print("Movie Duration IQR Range:", lower_bound, "-", upper_bound)
print("Number of Outliers:", outliers.count())
```

6. Insights based on Non-Graphical and Visual Analysis **(10 Points)**
6.1 Comments on the range of attributes
6.2 Comments on the distribution of the variables and relationship between them
6.3 Comments for each univariate and bivariate plot

ANS:-

```
print("\nMovie Duration Stats:\n", simulated_df[simulated_df['type'] == 'Movie']['duration_int'].describe())
print("\nTV Show Season Stats:\n", simulated_df[simulated_df['type'] == 'TV
Show']['duration_int'].describe())
print("\nGenre Popularity:\n", simulated_df['listed_in'].value_counts())
print("\nRating Distribution:\n", simulated_df['rating'].value_counts())
print("\nRelease Year Distribution:\n", simulated_df['release_year'].value_counts().sort_index())
```

7. Business Insights.

ANS:-

```
print("\nContent Type %:\n", simulated_df['type'].value_counts(normalize=True) * 100)
print("\nTop Countries for Content:\n", simulated_df['country'].value_counts().head())
print("\nFamily Content Gap: Ratings Distribution\n", simulated_df['rating'].value_counts())
```

8. Recommendations.

ANS:-Print Recommendations from data

```
recommendations = [
    "1. Produce more TV Shows to improve user engagement.",
    "2. Create more regional content for India, UK, Brazil.",
    "3. Expand beyond Dramas and Comedies into untapped genres.",
    "4. Fill missing metadata (director, cast, country, rating).",
    "5. Launch content during global holiday seasons.",
    "6. Develop family-oriented and educational titles.",
    "7. Focus on 90-120 min movies and short-season TV shows.",
    "8. Use localized strategy for mobile-first countries."
]
for r in recommendations:
    print(r)
```